

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

Errors in Haskell (the rough way)

Trees

- Binary Trees

- Conversion between Trees and Lists

- Binary Search Trees

- Printing trees

Propositional reasoner

Errors

Errors in Haskell

How can we deal with an illegal operation, e.g. division by 0?

Example (Legal v. illegal operations in Haskell)

```
divR :: Int -> Int -> Int
```

```
divR a b = div a b
```

```
*Main> divR 10 2
```

```
5
```

```
*Main> divR 10 0
```

```
*** Exception: divide by zero
```

Haskell errors

How to introduce Haskell errors

`error` + a string describing the error

Example (Errors in Haskell)

```
divRE a b | b == 0      = error "div by 0, dude"  
          | otherwise = div a b
```

```
*Main> divRE 1 0  
*** Exception: div by 0, dude  
CallStack (from HasCallStack):  
  error, called at errdivRE.lhs:2:25 in main:Main
```

Note

Expression evaluation is stopped and the program crashes.
Better options are available (we'll meet them down the road)

Haskell errors

Example (Dealing with errors in a lazy language)

```
divRE a b | b == 0      = error "div by 0, dude"  
          | otherwise = div a b
```

```
Main> head [divRE 2 2, divRE 2 0]  
1
```

Catching errors

Deferred until discussing monads.

Binary trees as a new type

Example (Binary tree: each node has at most 2 sub-trees)

```
data Tree a = Empty
            | Node a (Tree a) (Tree a)
            deriving (Show)
```

Example

```
t1=Node "A"
      (Node "B"
           Empty
           (Node "C"
                (Node "D" Empty Empty)
                Empty))
      Empty :: Tree [Char]
```

Binary trees as a new type

Example (Tree height: equation(s) to cover each case)

```
h2 :: Tree a -> Int
h2 Empty          = 0
h2 (Node _ l r)   = 1 + max (h2 l) (h2 r)
```

Example (Computing height)

```
*Main> h2 t1
4
```

Example (Tree height, alternative implementation)

```
height :: Tree a -> Int
height t = case t of
  Empty      -> 0
  Node _ l r -> 1 + max (height l) (height r)
```

Binary trees as a new type

Example (Reflecting a tree)

```
reflect :: Tree a -> Tree a
reflect t = case t of
  Empty      -> Empty
  Node v l r -> Node v
                                (reflect r)
                                (reflect l)
```

Binary trees as a new type

Example (Reflecting a tree)

```
g1 = Node 1
      (Node 2
        (Node 4 Empty Empty)
        (Node 5 Empty Empty))
      (Node 3
        (Node 6 Empty Empty)
        Empty) :: Tree Int
```

```
*Main> reflect g1
Node 1 (Node 3 Empty (Node 6 Empty Empty)) (Node
2 (Node 5 Empty Empty) (Node 4 Empty Empty))
```

Binary trees as a new type

Example (Reflecting a tree)

```
*Main> g1 == reflect (reflect g1)
No instance for (Eq (Tree Int))
arising from a use of '=='
```

Tree to list

Example (Preorder)

```
preord t = case t of
  Empty      -> []
  Node v l r -> (v:preord l) ++ (preord r)
```

```
*Main> preord g1
[1,2,4,5,3,6]
```

Tree to list

Remark

Append (++) is expensive: $O(n^2)$ CONS operations

Example (Preorder with a "collecting" parameter)

```
preordA t vs = case t of
  Empty      -> vs
  Node v l r -> v:preordA l (preordA r vs)
```

```
*Main> preordA g1 []
[1,2,4,5,3,6]
```

Binary search tree

Binary Search Trees v. Association Lists

1. both store info as (key,value) pairs
2. search time: $O(\log n)$ (v. $O(n)$ for AL)
3. type of key: `Ord` for BST (v. `Eq` suffices for AL)

Binary search tree

Example (Searching in a binary search tree)

```
data Tree a = Empty
            | Node a (Tree a) (Tree a)
            deriving (Show)

searchB :: Ord t1 => t1 -> Tree (t1, t) -> t
searchB s t = case t of
    Empty          -> error "key missing"
    Node (k,v) l r -> case (s == k) of
        True  -> v
        False -> case s < k of
            True  -> searchB s l
            False -> searchB s r
```

Binary search tree

Example (Searching in a binary search tree)

```
dict1 = Node ("dos", 2)
        (Node ("cinque", 5) Empty Empty)
        (Node ("tres", 3)
            (Node ("quattro", 4) Empty Empty)
            (Node ("uno", 1) Empty Empty))
```

```
*Main> searchB "tres" dict1
```

```
3
```

Binary search tree

Example (Inserting nodes in a binary search tree)

```
insertB :: Ord t1 =>
  t1 -> t -> Tree (t1, t) -> Tree (t1, t)
insertB nk nv t = case t of
  Empty          -> Node (nk, nv) Empty Empty
  Node (k,v) l r -> case nk == k of
    True  -> error "Key already present"
    False -> case nk < k of
      True  -> Node (k,v) (insertB nk nv l) r
      False -> Node (k,v) l (insertB nk nv r)

*Main> insertB "seis" 6 dict1
Node ("dos",2) (Node ("cinque",5) Empty Empty) ...
```

Side effect - printing on the screen

Example (Print trees: rely on what Haskell `Show` provides)

```
data Mytree t = Empty
              | Node t (Mytree t) (Mytree t)
              deriving (Show)

t3 = Node 3
      (Node 1 Empty Empty)
      (Node 4 Empty (Node 5 Empty Empty))

*Main> t3
Node 3 (Node 1 Empty Empty) (Node 4 Empty
(Node 5 Empty Empty))
```

Side effect - printing on the screen

Example (Write your own `show` to pretty print trees)

```
data Mytree t = Empty
               | Node t (Mytree t) (Mytree t)

instance (Show t) => Show (Mytree t) where
  show = pT 0
  where
    pInd i = [ ' ' | k <- [1..i] ]
    pT i Empty          = pInd i ++ "." ++ "\n"
    pT i (Node v l r) = pInd i ++
                        (show v) ++ "\n" ++
                        (pT (i+3) l) ++
                        (pT (i+3) r)
```

Side effect - printing on the screen

Example (Write your own `show` to pretty print trees)

```
t3 = Node 3  
      (Node 1 Empty Empty)  
      (Node 4 Empty (Node 5 Empty Empty))
```

```
*Main> t3
```

```
3  
  1  
    .  
    .  
  4  
    .  
  5  
    .  
    .
```

Side effect - printing on the screen

Remark

In solution #2, we have specified the constraint `(Show t)`. So, when overloading `show` for trees, we can use `(show v)` and the tree printing function will work regardless the actual type of the values stored in the tree (if they are showable).

Propositional logic

Components

1. atoms: $a, b, \dots$
2. connectors: $\neg, \wedge, \vee$ (printed as $\sim, \&, |$)

Example (A type for representing propositions)

```
data Prop = Atom String
          | Neg Prop
          | Or Prop Prop
          | And Prop Prop
```

Example (Representing proposition $a \wedge (b \vee c)$)

```
ex1 = And (Atom "a") (Or (Atom "b") (Atom "c"))
```

Will be printed as: $a \& (b | c)$

Propositional logic

Example (A type for representing propositions)

```
data Prop = Atom String
          | Neg Prop
          | Or Prop Prop
          | And Prop Prop
```

Example (Representing Implication: $p \rightarrow q \equiv \neg p \vee q$)

```
imply p q = Or (Neg p) q
```

Propositional logic

Example (Pretty printing propositions)

```
instance Show Prop where
  show = prtP 0 where
    prec oper = case oper of -- op precedence
      Atom _   -> 4
      Neg _    -> 3
      And _ _  -> 2
      Or  _ _  -> 1
```

Propositional logic

Example (Pretty printing propositions)

```
prtP cpl prop =  
  let  
    ppl = prec prop  
    prnpar = case prop of --with  no parentheses  
      Atom p    -> p  
      Neg p     -> "~" ++ prtP ppl p  
      And p q   -> prtP ppl p ++ " & " ++ prtP ppl q  
      Or p q    -> prtP ppl p ++ " | " ++ prtP ppl q  
  in  
    if (ppl > cpl) then prnpar  
      else "(" ++ prnpar ++ ")"
```

Negative Normal Form (NNF): negation on atoms only

Conversion to NNF: move negation inwards

- (1) $\neg\neg p$ replaced by p
- (2) $\neg(p \wedge q)$ replaced by $\neg p \vee \neg q$
- (3) $\neg(p \vee q)$ replaced by $\neg p \wedge \neg q$

Example

`tonnf prop = case prop of`

`Atom a -> Atom a --1`

`Neg (Atom a) -> Neg (Atom a) --2`

`Neg (Neg p) -> tonnf p --3`

`Neg (And p q) -> tonnf (Or (Neg p) (Neg q)) --4`

`Neg (Or p q) -> tonnf (And (Neg p) (Neg q)) --5`

`And p q -> And (tonnf p) (tonnf q) --6`

`Or p q -> Or (tonnf p) (tonnf q) --7`

NNF and CNF (Conjunctive/Clausal Normal Form)

Example (Obtaining NNF: $\neg(p \rightarrow q) \equiv_{NNF} p \wedge \neg q$)

```
p      = Atom "p"  
q      = Atom "q"  
nil    = Neg (imply p q)
```

```
*Main> tonnf nil  
p & ~q
```

NNF and CNF (Conjunctive/Clausal Normal Form)

Example (Obtaining NNF: $\neg(p \rightarrow q) \equiv_{NNF} p \wedge \neg q$)

```
tonnf prop = case prop of
  Atom a          -> Atom a                --1
  Neg (Atom a)    -> Neg (Atom a)          --2
  Neg (Neg p)     -> tonnf p               --3
  Neg (And p q)   -> tonnf (Or (Neg p) (Neg q)) --4
  Neg (Or p q)    -> tonnf (And (Neg p) (Neg q)) --5
  And p q         -> And (tonnf p) (tonnf q) --6
  Or p q          -> Or (tonnf p) (tonnf q)  --7

tonnf (Neg (imply (Atom "p") (Atom "q"))) =      --def of imply
tonnf (Neg (Or (Neg (Atom "p")) (Atom "q"))) =    --5
tonnf (And (Neg (Neg (Atom "p"))) (Neg (Atom "q"))) = --6
And (tonnf (Neg (Neg (Atom "p")))) (tonnf (Neg (Atom "q"))) = --3
And (tonnf (Atom "p")) (tonnf (Neg (Atom "q"))) = --2
And (Atom "p") (Neg (Atom "q"))
```

NNF and CNF (Conjunctive/Clausal Normal Form)

Definition (CNF of a sentence)

p is in CNF if it is of form $p_1 \wedge \dots \wedge p_n$, where p_i is a disjunction of literals (atoms or negated atoms).

Conversion from NNF to CNF

$p \vee (q \wedge r)$ replaced by $(p \vee q) \wedge (p \vee r)$

$(q \wedge r) \vee p$ replaced by $(q \vee p) \wedge (r \vee p)$

From NNF to CNF

Conversion from NNF to CNF

$p \vee (q \wedge r)$ replaced by $(p \vee q) \wedge (p \vee r)$

$(q \wedge r) \vee p$ replaced by $(q \vee p) \wedge (r \vee p)$

Example (Distributivity)

$\text{dist } p \ (\text{And } q \ r) = \text{And } (\text{dist } p \ q) \ (\text{dist } p \ r)$

$\text{dist } (\text{And } q \ r) \ p = \text{And } (\text{dist } q \ p) \ (\text{dist } r \ p)$

$\text{dist } p \ q = \text{Or } p \ q$

From NNF to CNF

Example (Obtaining CNF from NNF)

```
tocnf prop = case prop of
  And p q -> And (tocnf p) (tocnf q)
  Or p q   -> dist (tocnf p) (tocnf q)
  _        -> prop
```

From NNF to CNF

Example $(\neg(\neg p \vee q) \vee \neg(p \wedge q)) \equiv_{NNF} (p \wedge \neg q) \vee (\neg p \vee \neg q)$

```
f2 = Or (Neg (Or (Neg (Atom "p")) (Atom "q")))
      (Neg (And p q))
```

```
*Main> tonnf f2
p & ~q | (~p | ~q)
```

From NNF to CNF

Example (Getting CNF: $\neg(\neg p \vee q) \vee \neg(p \wedge q) \equiv_{CNF}$
 $(p \vee (\neg p \vee \neg q)) \wedge (\neg q \vee (\neg p \vee \neg q)))$)

```
*Main> tonnf f2
```

```
p & ~q | (~p | ~q)
```

```
*Main> tocnf (tonnf f2)
```

```
(p | (~p | ~q)) & (~q | (~p | ~q))
```

Definition (Tautology)

Tautology = a sentence which is **always true**.

Testing whether a sentence is a tautology

1. put it into CNF ($p_1 \wedge \dots \wedge p_n$) and check if p_i is a tautology for every i between 1 and n
2. a disjunctive formula $q_1 \vee \dots \vee q_m$ is a tautology iff there is a literal which appears both as an atom and as its negation . . .
3. . . so separately collect all positive and all negative atoms in that formula and check if the two sets are not disjoint

Testing if a formula is a tautology

Example (Collect positive atoms)

```
getpos prop = case prop of
  Atom a          -> [a]
  Neg (Atom _)    -> []
  Or p q          -> (getpos p) ++ (getpos q)
  _               -> error "proposition not in CNF"
```

Example (Collect negative atoms)

```
getneg prop = case prop of
  Atom _          -> []
  Neg (Atom a)    -> [a]
  Or p q          -> (getneg p) ++ (getneg q)
  _               -> error "proposition not in CNF"
```

Testing if a formula is a tautology

Example (Is it a tautology?)

```
istaut prop = case prop of
  And p q -> (istaut p) && (istaut q)
  _        -> not (null [ a | a <- (getpos prop),
                               elem a (getneg prop)])
```

Example (Is $((p \vee q) \vee \neg p)$ it a tautology?)

```
p      = Atom "p"
np     = Neg p
q      = Atom "q"
nq     = Neg q
taul   = Or (Or p q) np

*Main> istaut (tocnf (tonnf taul))
True
```

Example

Why do we need a tautology checker?

Summary

- ▶ `error` causes evaluation to stop and crash. Avoid!
- ▶ Constructors can help handling heterogenous structures (e.g. trees)
- ▶ Side effects help pretty printing trees
- ▶ It is easy to implement a propositional reasoner in functional manner.